



Ανάπτυξη και Ασφάλεια Πληροφοριακών Συστημάτων

Τεχνολογίες & Υπηρεσίες Διαδικτύου

Εργασία 1 - Mininet

2025-2026

Ελένη Κεχριώτη

f3312503

Ο κύριος στόχος της παρούσας εργασίας ήταν η πρακτική εφαρμογή των αρχών του Software Defined Networking (SDN), μέσω της υλοποίησης ενός απλού Firewall σε ένα εικονικό περιβάλλον δικτύου. Συγκεκριμένα, χρησιμοποιήθηκαν τα εργαλεία Mininet Network Emulator και POX OpenFlow Controller για τη δημιουργία μιας custom τοπολογίας και τον προγραμματισμό του Switch (S1) με συγκεκριμένους κανόνες ασφαλείας (Flow Rules).

Αρχιτεκτονική Συστήματος

Το υλοποιημένο σύστημα βασίζεται στην αρχιτεκτονική SDN, η οποία διαχωρίζει το επίπεδο ελέγχου (Control Plane) από το επίπεδο δεδομένων (Data Plane). Στη παρούσα εργασία το ρόλο του επιπέδου ελέγχου τον έχει ο POX Controller, καθώς αυτό είναι που εφαρμόζει τη λογική του firewall και στέλνει τους κανόνες στο switch. Τον ρόλο του επιπέδου δεδομένων τον έχει το switch, καθώς αυτό απλώς ελέγχει τον πίνακα ροής και εκτελεί την αντίστοιχη ενέργεια.

- Mininet (Network Emulator)

Το Mininet χρησιμοποιήθηκε για την εικονική δημιουργία της τοπολογίας του δικτύου. Η custom τοπολογία ορίστηκε στο αρχείο myTopo.py.

- POX (OpenFlow Controller)

Ο POX Controller αποτελεί το Control Plane του συστήματος. Εκτελέστηκε απομακρυσμένα και είναι υπεύθυνος για:

- τη σύνδεση και διαχείριση του switch S1.
- την υλοποίηση της λογικής του Firewall.
- την εγκατάσταση των κανόνων ροής στον πίνακα ροής του switch μέσω μηνυμάτων.

- OpenFlow Switch (S1)

Το switch S1 λειτουργεί ως το Data Plane (Επίπεδο Δεδομένων). Η λειτουργία του βασίζεται αποκλειστικά στους κανόνες που λαμβάνει από τον POX Controller. Όταν λαμβάνει ένα πακέτο, το switch ελέγχει τον Flow Table για ταιριαστό κανόνα.

- Αν βρει κανόνα, εκτελεί την αντίστοιχη ενέργεια (ACCEPT/DROP/FLOOD).
- Αν δεν βρει κανόνα, στέλνει το πακέτο στον Controller για καθοδήγηση. Οι κανόνες του firewall είναι μόνιμοι, ώστε το switch να μη χρειάζεται να ρωτάει τον controller.

Υλοποίηση Τοπολογίας και Κανόνων Firewall

- Δομή της Τοπολογίας

Η τοπολογία υλοποιήθηκε στο αρχείο myTopo.py και περιλαμβάνει:

- 1 Switch (S1)

```
s1 = self.addSwitch("s1")
```

- 4 Hosts (H1, H2, H3, H4)

```
h1 = self.addHost("h1", mac="00:00:00:00:00:01", ip="10.0.2.10/24")
```

```
h2 = self.addHost("h2", mac="00:00:00:00:00:02", ip="10.0.2.20/24")
h3 = self.addHost("h3", mac="00:00:00:00:00:03", ip="10.0.2.30/24")
h4 = self.addHost("h4", mac="00:00:00:00:00:04", ip="10.0.2.40/24")
```

Σύνδεση όλων των hosts απευθείας στο s1

```
self.addLink(h1, s1)
self.addLink(h2, s1)
self.addLink(h3, s1)
self.addLink(h4, s1)
```

Ο S1 ρυθμίστηκε ώστε να ελέγχεται από έναν Remote Controller στη διεύθυνση 127.0.0.1:6666.

```
net = Mininet(
    topo=topo,
    controller=lambda a: RemoteController(a, ip=CONTROLLER_IP,
    port=CONTROLLER_PORT)
)
```

Παρακάτω φαίνεται το screenshot, όπου δημιουργήθηκε το εικονικό δίκτυο.

```
csuser@csuser-virtualbox:~/Desktop/mcis$ sudo python3 myTopo.py
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1 ...

*** Mininet topology started with Remote POX Controller at 127.0.0.1:6
666 ***

*** Starting CLI:
```

- Οι Κανόνες του Firewall

Οι κανόνες του firewall υλοποιήθηκαν ως Flow Rules στο αρχείο myController.py. Η σωστή λειτουργία τους εξαρτάται από την προτεραιότητα που έχουν, όπου οι ειδικοί κανόνες (ACCEPT/DROP) πρέπει να έχουν υψηλότερη προτεραιότητα από τους γενικούς κανόνες.

A/A	Κανόνας (Rule)	Match Criteria	Actions (Ενέργεια)	Priority
1.	ACCEPT ARP	dl_type=0x0806 (ARP)	OFPP_FLOOD	100
2.	ACCEPT UDP	IPv4, nw_proto=17 (UDP)	OFPP_FLOOD	90
3.	ACCEPT HTTP (H1→H4)	TCP, tp_dst=80, nw_src=H1, nw_dst=H4	OFPP_FLOOD	80
4.	ACCEPT HTTP (H4→H1)	TCP, tp_src=80, nw_src=H4, nw_dst=H1	OFPP_FLOOD	80
5.	DROP λοιπά TCP	IPv4, nw_proto=6 (TCP)	No Action (DROP)	70

Παρακάτω φαίνεται η δημιουργία των κανόνων και η επιτυχής εγκατάσταση τους στο switch s1.

```

csuser@csuser-virtualbox:~$ cd /home/csuser/pox
csuser@csuser-virtualbox:~/pox$ ./pox.py openflow.of_01 --port=6666 mi
sc.myController
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:misc.myController:Firewall POX controller running (Python 3.8 com
patible)
WARNING:version:POX requires one of the following versions of Python:
3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported versio
n.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
INFO:misc.myController:Flow installed with priority 100
INFO:misc.myController:Flow installed with priority 90
INFO:misc.myController:Flow installed with priority 80
INFO:misc.myController:Flow installed with priority 80
INFO:misc.myController:Flow installed with priority 70
INFO:misc.myController:Firewall rules installed on switch 1

```

- Λειτουργία και Ανάλυση του Κώδικα Firewall

Η λογική του `myController.py` βασίζεται στην κλάση `Firewall` και στη μέθοδο `install_firewall_rules`, η οποία στέλνει τα μηνύματα `ofp_flow_mod` στο switch μόλις αυτό συνδεθεί.

Για την ταυτοποίηση των πακέτων, χρησιμοποιήθηκαν τα ακόλουθα πεδία των OpenFlow Match Criteria:

- **dl_type (Data Link Type):** Χρησιμοποιείται για την αναγνώριση πρωτοκόλλων επιπέδου 2 και 3.
- **nw_proto (Network Protocol):** Χρησιμοποιείται για την αναγνώριση πρωτοκόλλων επιπέδου 3 (όταν το data link είναι IP).
- **tp_dst / tp_src (Transport Port):** Χρησιμοποιείται για την αναγνώριση θυρών επιπέδου 4 (π.χ., TCP/UDP).

Οι κανόνες που υλοποιήθηκαν ήταν οι εξής:

1. **ARP:** Δημιουργεί ένα αντικείμενο match που αναζητά πακέτα όπου το data link type είναι 0x0806. Ο κωδικός αυτός χρησιμοποιείται για την αναγνώριση πακέτων ARP. Για να εγκατασταθεί ο κανόνας ροής (η 2η εντολή) χρησιμοποιείται το κριτήριο αυτό που φτιάξαμε με ενέργεια flood για να προωθηθεί το πακέτο σε όλες τις θύρες αν αντιστοιχιστεί σωστά το κριτήριο με το πακέτο. Ο κανόνας αυτός έχει την υψηλότερη προτεραιότητα στο σύστημα γιατί το ARP πρέπει πάντα να περνάει για να λειτουργήσει σωστά το δίκτυο.

```
arp_match = of.ofp_match(dl_type=0x0806)
self.send_flow_mod(match=arp_match, action=action_flood, priority=100)
```

2. **UDP:** Δημιουργεί ένα αντικείμενο match που αναζητά πακέτα όπου το dl_type=0x0800 (IPv4) και το nw_proto=17. Ο κωδικός 17 χρησιμοποιείται για την αναγνώριση πακέτων UDP. Για να εγκατασταθεί ο κανόνας ροής (η 2η εντολή) χρησιμοποιείται το κριτήριο αυτό που φτιάξαμε με ενέργεια normal για να προωθηθεί το πακέτο μόνο στη διεύθυνση προορισμού στη σωστή θύρα (δηλαδή να επιτραπεί) αν αντιστοιχιστεί σωστά το κριτήριο με το πακέτο. Το switch αυτομάτως επιστρέφει στη λειτουργία του παραδοσιακού switch L2/L3 και χρησιμοποιεί την πληροφορία του header (MAC διεύθυνση) για να στείλει το πακέτο μόνο στη σωστή θύρα εξόδου. Ο κανόνας αυτός έχει υψηλή προτεραιότητα (90) στο σύστημα γιατί η κίνηση UDP επιτρέπεται για να διασφαλιστεί η διέλευση.

```
udp_match = of.ofp_match(dl_type=0x0800, nw_proto=17)
udp_action = of.ofp_action_output(port = of.OFPP_NORMAL)
self.send_flow_mod(match=udp_match, action=udp_action, priority=90)
```

3. **HTTP:** Δημιουργούν δύο αντικείμενα match που αναζητούν πακέτα τα οποία είναι:
 - a. dl_type=0x0800 (IPv4) και nw_proto=6 (TCP).
 - b. Προς H4 (Κανόνας 1 - Αίτημα): Αναζητά πακέτα όπου η πηγή (nw_src) είναι ο H1_IP, ο προορισμός (nw_dst) είναι ο H4_IP και η θύρα προορισμού (tp_dst) είναι η HTTP_PORT (80).

- c. Προς H1 (Κανόνας 2 - Απάντηση): Αναζητά πακέτα όπου η πηγή (nw_src) είναι ο H4_IP, ο προορισμός (nw_dst) είναι ο H1_IP και η θύρα πηγής (tp_src) είναι η HTTP_PORT (80).

Για να εγκατασταθούν οι κανόνες ροής (οι 2ες εντολές) χρησιμοποιείται το αντίστοιχο κριτήριο που φτιάχτηκε με ενέργεια flood για να προωθηθούν τα πακέτα σε όλες τις θύρες (δηλαδή να επιτραπεί η αμφίδρομη επικοινωνία) αν αντιστοιχισθούν σωστά τα κριτήρια με τα πακέτα. Οι κανόνες αυτοί έχουν υψηλή προτεραιότητα (80) για να υπερισχύουν του γενικού κανόνα DROP για την υπόλοιπη TCP κίνηση. Η επιτυχία τους επιβεβαιώνει την ορθή υλοποίηση των Κανόνων 3 & 4 (ACCEPT HTTP H1 ↔ H4).

```
h1_h4_http_match = of.ofp_match(  
    dl_type=0x0800, nw_proto=6,  
    nw_src=H1_IP, nw_dst=H4_IP, tp_dst=HTTP_PORT  
)  
self.send_flow_mod(match=h1_h4_http_match, action=action_flood,  
priority=80)  
  
h4_h1_http_match = of.ofp_match(  
    dl_type=0x0800, nw_proto=6,  
    nw_src=H4_IP, nw_dst=H1_IP, tp_src=HTTP_PORT  
)  
self.send_flow_mod(match=h4_h1_http_match, action=action_flood,  
priority=80)
```

4. **TCP:** Αυτός ο κανόνας λειτουργεί ως η default πολιτική ασφαλείας για όλη την υπόλοιπη TCP κίνηση. Χρησιμοποιεί match που αναζητά πακέτα όπου το dl_type=0x0800 (IPv4) και το nw_proto=6 (TCP) και στέλνει ένα μήνυμα με κενή λίστα ενεργειών. Στο OpenFlow, η παράλειψη ενεργειών έχει ως αποτέλεσμα την απόρριψη (DROP) του πακέτου. Επειδή έχει χαμηλότερη προτεραιότητα (70) από τους ειδικούς κανόνες HTTP (80), επεξεργάζεται μόνο τα πακέτα που είναι TCP και δεν ταιριάζουν στους ειδικούς κανόνες, εξασφαλίζοντας έτσι ότι μόνο η επιτρεπόμενη κίνηση περνάει.

```
tcp_drop_match = of.ofp_match(dl_type=0x0800, nw_proto=6)  
self.send_flow_mod(match=tcp_drop_match, action=None, priority=70)
```

Αποτελέσματα Δοκιμών και Ανάλυση

- Έλεγχος Τοπολογίας

```
mininet> dump
<Host h1: h1-eth0:10.0.2.10 pid=3337>
<Host h2: h2-eth0:10.0.2.20 pid=3339>
<Host h3: h3-eth0:10.0.2.30 pid=3341>
<Host h4: h4-eth0:10.0.2.40 pid=3343>
<OVSSwitch s1: lo:127.0.0.1,s1-eth1:None,s1-eth2:None,s1-eth3:None,s1-eth4:None pid=3348>
<RemoteController c0: 127.0.0.1:6666 pid=3331>
```

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0
c0
```

Όπως φαίνεται στα screenshots, η τοπολογία δημιουργήθηκε επιτυχώς με 4 hosts (H1-H4) και 1 switch (S1). Όλοι οι hosts έλαβαν τις σωστές IP διευθύνσεις (10.0.2.X/24). Ο Remote Controller (c0) συνδέθηκε στο switch, επιβεβαιώνοντας τη βασική αρχιτεκτονική SDN.

- Έλεγχος ICMP/ARP

Το pingall απέτυχε πλήρως, κάτι που ήταν το αναμενόμενο, καθώς δεν υπάρχει ρητός κανόνας στο firewall που να επιτρέπει τη διέλευση μηνυμάτων ICMP. Τα πακέτα ICMP δεν ταιριάζανε σε κάποιον κανόνα και κατέληγαν στον controller στο `_handle_PacketIn` όπου και απορριπτόντουσαν.


```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X X X
h3 -> X X X
h4 -> X X X
*** Results: 100% dropped (0/12 received)
```

Το arp όμως δουλεύει κανονικά αφού ο h1 γνωρίζει και κρατάει τον πίνακα με τους άλλους hosts και τις διευθύνσεις τους.

```
mininet> h1 arp -n
```

Address	Hwtype	Hwaddress	Flags	Mask	Iface
10.0.2.20	ether	00:00:00:00:00:02	C		h1-eth0
10.0.2.40	ether	00:00:00:00:00:04	C		h1-eth0
10.0.2.30	ether	00:00:00:00:00:03	C		h1-eth0

- Έλεγχος UDP

Το αποτέλεσμα δείχνει ότι η κίνηση UDP πέρασε επιτυχώς μεταξύ H1 και H4 με bandwidth 10.5 Mbits/sec. Αυτό επιβεβαιώνει την ορθή λειτουργία του ACCEPT UDP.

```
mininet> iperfudp
*** Iperf: testing UDP bandwidth between h1 and h4
*** Results: ['10M', '10.5 Mbits/sec', '10.5 Mbits/sec']
```

- Έλεγχος TCP (HTTP & TCP DROP)

Σύμφωνα με τους κανόνες του firewall θα πρέπει να επιτρέπεται η επικοινωνία μεταξύ h1 και h4. Πράγματι, στο παρακάτω screenshot επιβεβαιώνεται ο κανόνας αυτός, αφού ο h1 συνδέθηκε επιτυχώς στον server του h4 και κατέβασε την σελίδα index.html.

```

mininet> h4 python3 -m http.server 80 &
mininet> h1 wget 10.0.2.40
--2025-11-26 23:58:48-- http://10.0.2.40/
Connecting to 10.0.2.40:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 285 [text/html]
Saving to: 'index.html'

index.html          0%[          ] 0  --.-KB/s
index.html        100%[=====] 285  --.-KB/s   in
 0s

2025-11-26 23:58:48 (18,7 MB/s) - 'index.html' saved [285/285]

```

Θα πρέπει όμως να μην επιτρέπεται οποιαδήποτε άλλη κίνηση.

Με την εντολή **h2 iperf -s &** θα ελέγξουμε ακριβώς αυτό. Εκκινούμε την εφαρμογή iperf σε λειτουργία server που τρέχει στο παρασκήνιο. Εδώ θα το χρησιμοποιήσουμε απλά για να δημιουργήσουμε μια τυπική σύνδεση TCP. Με την εντολή **h1 iperf -c 10.0.2.20** ορίζουμε το h1 ως πελάτη που ξεκινά τη σύνδεση και προσπαθεί να την δημιουργήσει προς τον h2

Ο h1 βλέπουμε ότι αποτυγχάνει να συνδεθεί στον h2, όπως μπορούσε πριν ο h1. Η αποτυχία της σύνδεσης iperf μεταξύ H1 και H2 επιβεβαιώνει την ορθή λειτουργία του Κανόνα 6 (DROP λοιπά TCP), καθώς η κίνηση αυτή δεν ταιριάζει με τους κανόνες.

```

mininet> h2 iperf -s &
mininet> h1 iperf -c 10.0.2.20
-----
Client connecting to 10.0.2.20, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
tcp connect failed: Connection timed out
[ 1] local 0.0.0.0 port 0 connected with 10.0.2.20 port 5001
mininet> h3 telnet 10.0.2.10 22
Trying 10.0.2.10...
telnet: Unable to connect to remote host: Connection timed out
mininet>

```

```

mininet> h2 wget --timeout=2 --tries=1 10.0.2.40
--2025-11-27 00:53:27-- http://10.0.2.40/
Connecting to 10.0.2.40:80... failed: Connection timed out.
Giving up.

```

- Εμφάνιση Κανόνων Flow Table

Η εντολή αυτή δεν μου έτρεχε στο mininet και μου εμφάνιζε μήνυμα ότι δεν έβρισκε το s1. Η εντολή **ovs-ofctl dump-flows s1** (που είναι ισοδύναμη με το **dpctl dump-flows s1**) εμφανίζει τον Flow Table του switch S1 και επιβεβαιώνει την σωστή εγκατάσταση των κανόνων του firewall απο τον controller. Παρατηρούμε ότι οι κανόνες είναι σωστά ιεραρχημένοι, με την σωστή προτεραιότητα που τους έχω δώσει και το σωστό action.

```
csuser@csuser-virtualbox:~/Desktop/mcis$ sudo ovs-ofctl dump-flows s1
[sudo] password for csuser:
 cookie=0x0, duration=889.091s, table=0, n_packets=44, n_bytes=1848, p
 riority=100,arp actions=FLOOD
 cookie=0x0, duration=888.945s, table=0, n_packets=4455, n_bytes=67332
 76, priority=90,udp actions=NORMAL
 cookie=0x0, duration=888.944s, table=0, n_packets=22, n_bytes=1628, p
 riority=70,tcp actions=drop
 cookie=0x0, duration=888.944s, table=0, n_packets=12, n_bytes=1056, p
 riority=80,tcp,nw_src=10.0.2.10,nw_dst=10.0.2.40,tp_dst=80 actions=FLO
 OD
 cookie=0x0, duration=888.944s, table=0, n_packets=12, n_bytes=1750, p
 riority=80,tcp,nw_src=10.0.2.40,nw_dst=10.0.2.10,tp_src=80 actions=FLO
 OD
```